

02-614 Week 3

String Algorithms

Aidan Jan

September 10, 2026

Edit Distance and Inexact Pattern Matching

Suppose we have two strings, a and b . The edit distance between a and b is defined as the number of follow operations needed to turn a into b , where the operations are:

1. Delete a character in a
2. Insert a character in a
3. Change a character in a

For example, $a = \text{"riddle"}$, $b = \text{"triple"}$

RIDDLE (start)
RIDLE (delete)
RIPLE (substitute)
TRIPLE (insert)

Note that all these actions are reversible. b can be transformed into a with the opposite operations. Additionally, the order of the changes does not matter.

Alignment

Because the actions are reversible and order-invariant, we can align the two strings:

-RIDDLE
X
TRIP-LE

We can write out the actions like the following, where - in the first word represents an insertion, a - in the second word represents a deletion, and an X represents a substitution.

Problem Statement

Given two strings a and b , compute `edit(a, b)` quickly or in small memory. Typically, we need small memory instead of fast time.

Dynamic Programming for Alignment

When considering alignment, we only care for the last character. The last character in each string can either:

- $x_k \leftrightarrow y_m$. (The two characters match.)
- $x_k \rightarrow -$. (The character in x is matched to a gap, or deletion.)
- $y_m \rightarrow -$. (The character in y is matched to a gap, or insertion.)

This acts as a recursive condition, where a given iteration depends entirely on previous iterations.

$ED(i, j) :=$ “the edit distance between the first i characters of x and the first j characters of y .”

$$ED(i, j) = \min \begin{cases} \text{match}(x_i, y_j) + ED(i-1, j-1) & \text{only if } x_i == y_j \\ \text{mismatch_cost}(x_i, y_j) + ED(i-1, j-1) \\ \text{deletion_cost}(x_i, y_j) + ED(i-1, j) \\ \text{insertion_cost}(x_i, y_j) + ED(i, j-1) \end{cases}$$

- The solution to the problem would be $ED(|x|, |y|)$
- The match score and mismatch, deletion, and insertion costs can be configured to weigh the selections differently. Typically the match score is negative and the costs are positive since we are measuring distance between x and y .
- These values can be computed efficiently using a dynamic programming table. (See 02-613 chapter on Dynamic Programming).

This algorithm will run in quadratic space and time since the table requires that amount of space.

- We can reduce this to linear space **if** we only need to know the edit distance (and not the alignment), because at any given iteration, we only need two rows. Therefore, we can store two rows in matrix at a time and reuse the rows. If we do this, we keep two columns OR two rows, whichever is smaller. This would lower space complexity to $O(\min(|X|, |Y|))$ from $O(|X||Y|)$.

Hirschberg’s Algorithm

In linear space, we can also compute all of the following from the DP table:

- Edit distance between every prefix of x and all y . (Last row of DP table is answer)
- Edit distance between all x and every prefix of y . (Last column of DP table is answer)
- **PrefixCosts**(x , i , y): Edit distance between a particular prefix of x and every prefix of y . (Some row in DP table is answer)
- **SuffixCosts**(x , i , y): Edit distance between a particular suffix of x and every suffix of y . (Reverse both x and y , then run algorithm.)

We name the last two because it will be convenient later. Note that they all take quadratic $O(|x||y|)$ time and linear $O(\min(|x|, |y|))$ space.

Hirschberg’s Algorithm attempts to compute the alignment in addition to edit distance, also in linear space.

Suppose we knew somehow where in y , $x[|x|/2]$ aligned. In this case, reducing space would be trivial, since now the algorithm, would become align the left, align the right, then combine them. The total space consumed more or less halves.

- The problem is that we don’t know where in y $x[|x|/2]$ would align.
- However, we can do *something* about it. Considering a given column (a character in x) on the DP table, we know for a fact at some point the backtracking arrows must pass through that column. Since the backtrack can only move left and up, we can discard the bottom left and top right of the matrix if we know where that path passes through.
- Conveniently, we have a quadratic-time, linear space algorithm to find exactly that.

Linear space algorithm:

```

def linearSpaceAlgorithm(x, y):
    yPrefix = PrefixCosts(x, |x| / 2, y)
    ySuffix = SuffixCosts(x, |x| / 2, y)
    best = infinity
    bestq = nil
    for q = 0 to |y|:
        editq = yPrefix[q] + ySuffix[q]
        if editq < best:
            bestq = q
            best = editq
    Add (|x| / 2, bestq) to global ArrowPath
    LinearSpaceAlign(x[1 : |x| / 2], y[1 : bestq])
    LinearSpaceAlign(x[|x| / 2 + 1 : ], y[bestq + 1 : ])

```

The algorithm essentially finds the best q , which represents the location in the column the backtrack path passes through. Then, it divides the problem into two and computes the two halves separately.

Linear Space Proof

Note that the algorithm is tail-recursive, as in, all recursive calls are at the end of the function. This means that we can deallocate all the space used for the other parts of the function when we do the recursive call. Therefore, space stays linear.

Quadratic Time Proof

$$T(m, n) < cmn + T(m/2, q) + T(m/2, n - q)$$

Or, the time for one run through the algorithm is equal to the cmn (time for finding q), plus the runtime of the two halves.

- If we assume that q is always half the length of the array, and $m = n$, then

$$T(n) \leq 2T\left(\frac{n}{2}\right) + cn^2$$

- By the master's theorem, it simplifies to n^2 .